Project: RaktKhoj — Find Blood in Minutes

Act as a senior full-stack engineer, UI/UX designer, database architect, and AI engineer. Build a complete, responsive, production-minded blood donor discovery and emergency coordination platform named **RaktKhoj**.

Use the attached UI screenshots as the primary visual reference. Recreate the design language, mobile layouts, multi-step emergency flow, donor matching cards, blood-bank search interface, and WhatsApp-style donor interaction as closely as practical. Do not create a generic hospital management dashboard.

1. Product vision

RaktKhoj helps people find nearby blood banks and potential blood donors during urgent situations.

The product should:

- Help users search for blood banks and potential donors near a location.
- Guide users through a simple, accessible emergency request flow.
- Match requests with suitable candidate donors using transparent rules.
- Notify consenting donors and let them accept or decline requests.
- Help requesters contact blood banks and responding donors.
- Support English and Telugu initially, with an architecture that allows more languages.
- Work beautifully on mobile devices, even on slower connections.

The core philosophy is: **Built for one thumb. Built for emergencies.**

Do not claim that a donor has been medically approved, that blood is available, or that a request is fulfilled unless this has actually been verified.

2. Technology stack

Frontend:

- React with Vite and TypeScript
- Tailwind CSS
- React Router
- Lucide React icons
- React Hook Form and Zod validation
- TanStack Query for API state
- Mapbox or an OpenStreetMap-compatible map solution

Backend:

- Node.js

- Express
- TypeScript
- REST API
- Zod request validation
- Secure authentication and role-based authorization

Database:

- PostgreSQL
- PostGIS for geographic searches
- Prisma ORM or an equivalent typed database layer

Supporting services:

- Official WhatsApp Business Platform integration, with a mock adapter for development
- SMS/email adapter where appropriate
- Redis and BullMQ for asynchronous notification jobs when needed
- An LLM API for optional conversational request assistance
- Docker for reproducible development

Use stable, compatible package versions. Provide environment-variable examples without exposing real secrets.

3. Visual design system

Follow the supplied RaktKhoj screenshots.

Brand:

- Product name: RaktKhoj
- Tagline: Find Blood in Minutes
- Primary colors: blood red, crimson and dark red
- Background: warm off-white with subtle blush tones
- Cards: white or light grey with rounded corners and subtle borders
- Buttons: strong red primary actions with accessible contrast
- Typography: clean, modern sans-serif
- Icons: simple, consistent line icons
- Use soft shadows and restrained gradients

Create a premium, polished interface with clear visual hierarchy, large tap targets, concise labels and minimal clutter.

Mobile must feel like the primary experience. On desktop, use a centered content layout and responsive navigation.

Do not use random gradients everywhere, oversized dashboards, excessive animations or placeholder lorem ipsum. Use meaningful empty states, skeleton loaders, success messages and error handling.

4. Main pages

A. Homepage

Create a landing page with:

- RaktKhoj logo and language selector
- Navigation menu
- Hero section: “Need blood right now?”
- Supporting text: “Get in touch with donors and blood banks near you, in minutes.”
- Primary CTA: “Find Blood Now”
- Secondary cards: “Become a Donor” and “Find Blood Camps”
- Platform statistics, clearly marked as sample data until verified
- Blood donation awareness banner
- How it works section
- Safety and eligibility information
- Footer with privacy, terms, emergency guidance and contact information

Never fabricate real registered-donor counts, available blood units or successful donation statistics.

B. Emergency blood request wizard

Build a polished five-step wizard inspired by the screenshots.

Step 1 — Select blood group:

- O+, O-, A+, A-, B+, B-, AB+, AB-
- Explain that the requested group is the patient's required group, subject to professional compatibility confirmation.
- Allow the user to continue without losing entered data.

Step 2 — Location:

- Select state, city and locality.
- Offer optional browser geolocation with explicit permission.
- Let users enter a location manually.
- Explain why location is needed.
- Never store precise location unless required and consented to.

Step 3 — Hospital or blood bank:

- Search registered facilities by name or location.
- Show distance, address, verification status and available contact details.
- Display availability only when supported by current verified data.
- Allow the user to select a facility or continue with an unverified facility clearly identified as such.

Step 4 — Patient and request details:

- Required blood component
- Number of units
- Urgency
- Patient name only when necessary
- Optional age and other necessary details
- Hospital contact details
- Keep sensitive medical information to a minimum.

Step 5 — Review and send:

- Summarize the request
- Explain which details will be shared
- Require consent before sharing information with donors
- Generate a clear, editable request message
- Submit the request and show its tracking ID

Include a visible progress indicator, back/next controls, validation, keyboard accessibility and persistent wizard state.

C. Blood-bank discovery

Build a search page with:

- Map and list views
- Search radius filters: 25 km, 50 km and 100 km
- Search by blood group and component
- Distance and contact information
- Verified versus unverified facility badges
- Last-updated timestamp for any inventory data
- Call and directions actions
- Clear empty states, such as “No confirmed availability within 50 km”
- A button to explore consenting donor matches when appropriate

Do not present mock inventory as real availability.

D. Donor discovery and matching

Show potential matches as cards with:

- First name or privacy-preserving display name
- Blood group
- Approximate distance
- Availability status
- Verification status
- Last verified profile date
- Contact options only when the donor has consented

Provide a “Search Donors Near You” action.

The system must not expose private phone numbers or precise home addresses publicly.

E. Request tracking

Create a tracking page with:

- Request ID
- Request status
- Number of candidate donors contacted
- Number of responses received
- Confirmed blood-bank contact information
- Timeline of status updates
- Ability to cancel or close the request

Statuses should include:
Draft, Open, Notifying, Responses Received, Coordinating, Fulfilled, Cancelled and Expired.

A donor accepting a request does not mean blood has been collected or transfused. Only authorized, verified confirmation may mark a request fulfilled.

F. Donor registration

Create a donor onboarding flow:

- Name
- Phone verification
- Blood group
- City and approximate location
- Consent to receive requests
- Availability preferences
- Last donation date, if voluntarily supplied
- Profile verification status

Do not automatically approve donors based on self-reported details.

Include a way to pause notifications, update information and delete an account.

G. Donor response experience

Create a mobile-first response interface inspired by the supplied WhatsApp screenshots.

A donor receives a request notification with minimal necessary information and can:

- Accept the request for further coordination
- Decline the request
- Indicate unavailability
- Review relevant safety guidance

If the donor accepts, ask appropriate screening questions only as a preliminary filter. Do not declare the donor medically eligible. Final eligibility and compatibility must be confirmed by qualified blood-bank professionals.

H. Admin dashboard

Provide a protected administrative interface for:

- Reviewing donor and facility verification requests
- Managing fraudulent or abusive requests
- Reviewing reported data errors
- Viewing request volumes and delivery failures
- Auditing administrative actions
- Managing supported languages and notification settings

Do not allow administrators to view unnecessary sensitive health information.

5. Database schema

Create migrations and typed models for the following entities:

Users:

- id
- name
- phone
- role
- account_status
- created_at
- updated_at

DonorProfiles:

- user_id
- blood_group
- approximate_location
- availability_status
- last_donation_date, nullable
- verification_status
- notification_consent
- verified_at

BloodRequests:

- id
- requester_id
- blood_group
- component
- units_requested
- hospital_id, nullable
- approximate_location
- urgency
- status

- consent_record
- created_at
- expires_at

BloodBanks:

- id
- name
- address
- location
- phone
- verification_status
- source
- last_verified_at

BloodBankInventory:

- id
- blood_bank_id
- component
- blood_group
- units_available
- source
- last_updated_at

Only implement inventory records when a legitimate data source exists. Mark demo data as simulated.

DonorMatches:

- id
- request_id
- donor_id
- distance_km
- match_reason
- notification_status
- response_status
- created_at

NotificationLogs:

- id
- request_id
- donor_id
- channel
- delivery_status
- provider_message_id
- created_at

ConsentRecords and AuditLogs:

- Store consent versions, relevant timestamps and necessary security audit information.
- Avoid logging sensitive medical details or authentication secrets.

Add foreign keys, indexes, unique constraints and appropriate transaction handling.

6. Matching engine

Implement a deterministic matching service.

Inputs:

- Requested blood group
- Requested component
- Search location and radius
- Donor's declared blood group
- Donor's current availability
- Profile verification status
- Recorded donation history, where supplied
- Notification consent

Processing:

1. Validate the request.
2. Exclude inactive, unverified or non-consenting profiles as appropriate.
3. Query nearby candidate donors using PostGIS.
4. Apply configurable preliminary matching rules.
5. Sort candidates using distance and relevant availability signals.
6. Create match records.
7. Enqueue notifications with duplicate prevention.
8. Record responses and update the request status.

Do not hard-code a simplistic universal blood compatibility matrix and treat it as medical authority. Use approved component-specific rules reviewed by qualified transfusion professionals. Rh compatibility, component type, antibodies and clinical circumstances can affect compatibility.

Use a configurable search radius and pagination. Explain why each candidate was included.

7. REST API

Implement authenticated, validated endpoints such as:

POST /api/auth/register
POST /api/auth/verify-phone
GET /api/banks/nearby
GET /api/banks/
POST /api/requests
GET /api/requests/
GET /api/requests//matches

POST /api/requests//cancel
POST /api/donors/profile
PATCH /api/donors/availability
POST /api/matches//respond
GET /api/requests//timeline
POST /api/webhooks/whatsapp
GET /api/admin/verifications

Use pagination, structured errors, authorization checks, request validation, rate limits and audit logs.

Never trust client-supplied distances, donor eligibility, verification status or request ownership. Calculate and validate important fields on the server.

8. WhatsApp integration

Create a notification-provider abstraction.

Development:

- Implement a mock WhatsApp provider.
- Show simulated outgoing notifications and incoming accept/decline events.
- Include a developer-only inbox for testing.
- Label all simulated activity clearly.

Production:

- Integrate the official WhatsApp Business Platform.
- Respect user opt-in, applicable template requirements, rate limits and messaging policies.
- Verify webhook signatures.
- Store provider message IDs and delivery status.
- Handle retries, failures and duplicate webhook events.
- Provide SMS or other appropriate fallback channels.

Do not claim that real WhatsApp messages were sent unless the provider confirms delivery.

9. Optional AI assistant

Build an AI assistant that helps users describe a blood request in natural language, including Telugu or English.

Example:
“My relative needs B-positive blood near Warangal.”

The assistant should:

- Extract the requested blood group and location into a structured object.
- Ask follow-up questions for missing information.

- Generate a draft request message.
- Explain how to find nearby blood banks.
- Never submit a request without explicit user confirmation.
- Never invent donor availability or blood-bank inventory.
- Never make medical diagnoses or independently approve donor eligibility.

Use an LLM with structured outputs, strict server-side validation, timeouts and error handling. The matching engine must remain deterministic. The core application must still work if the AI provider is unavailable.

10. Privacy, security and safety

Implement:

- HTTPS in production
- Secure authentication and session/token handling
- Role-based access control
- Rate limiting and anti-spam protection
- Consent-based contact sharing
- Minimal collection of personal and health information
- Retention and deletion policies
- Secure secrets management
- Verified facility and donor status
- Request expiry and notification deduplication
- Audit logs for privileged actions
- Clear reporting and abuse-handling workflows

Use synthetic data for development. Do not send real emergency notifications or use real patient data in demos.

Include a disclaimer that the platform coordinates potential donors and blood banks but does not replace medical professionals or guarantee blood availability.

11. Testing and quality

Write automated tests for:

- Authentication and authorization
- Blood-request validation
- Geographic search
- Matching logic
- Consent enforcement
- Donor acceptance and rejection
- Request cancellation and expiration
- Duplicate notification prevention
- Webhook verification
- Error handling and API failures

Create unit tests for the matching engine and integration tests for the core request workflow.

12. Deliverables

Build a complete runnable project with:

- Responsive React frontend
- Express backend
- PostgreSQL/PostGIS schema and migrations
- Seed script using clearly simulated data
- Working API integration
- Mock notification provider
- Optional AI assistant
- Environment-variable example file
- Docker Compose development setup
- README with setup and deployment instructions
- API documentation
- Tests
- A list of features that are simulated versus genuinely integrated

13. Development process

Work incrementally.

First, inspect the project directory and existing files. Preserve useful existing code.

Second, establish the design system, routing, database schema and application structure.

Third, implement the homepage and five-step emergency request flow.

Fourth, implement the backend, geographic matching, donor response workflow and request tracking.

Fifth, integrate mock notifications, then optional real provider integrations.

Finally, add tests, documentation, accessibility improvements and deployment configuration.

Do not stop after generating a plan or a static UI. Implement the actual working features, run available tests, fix errors and clearly report anything that remains incomplete.

Success criteria: A user can submit a simulated emergency request, find nearby synthetic blood-bank records, retrieve candidate donor matches, simulate notifications, record donor responses and track the request through the complete workflow in a responsive application.